Pricing a developed and an emerging insurance market

IAA AI-Assisted Hackathon 2026: final write-up

Krishna Kumar Shrestha · solo team

28 October 2026

	Week 1: developed market	Week 2: emerging market
Data	2021-24, 163,150 claims; 626,639 rows to price	2023-24, 9,258 claims; 209,063 rows to price
Submitted file	geometric mean of six experts, recalibrated per line	v13: ensemble ranking, segment levels read from the leaderboard
Public / private D^2	0.12851 / **0.13830**	**0.17172** / pending (v12, public 0.17095, also selected)

Both weeks are scored on the Tweedie deviance D^2 with power 1.5, pooled over the three lines. My first Week 2 file, a plain LightGBM, scored 0.12519. The work rests on three choices. I spent time on the target before building any model. In the new market I started from what the developed market already knew and let the local data correct it. And I kept a segment's level separate from the ranking inside it, because the 2025 book is too different from the training years for a back-test to say what the levels should be.

1 Data and target

Both weeks go through the same scripts (read, clean, build features), and each one has an executed notebook next to it as evidence. Every fix is logged with the rows it changed. I dropped no rows and imputed nothing. The audit found customer ids that had lost their leading zeros (123,664 rows), exposure recorded in days or on a 360-day basis (13,880 and 595 rows in the two weeks), sums insured entered in units instead of thousands (3,527 and 349), a case reserve typed with an extra digit (11.9m instead of 1.19m), and impossible years licensed, which I restored from the same policy's other years (3,717 rows). The last one found, after Week 1 had been submitted, was 604 claims stored twice under a second id. They had inflated 2022 claim counts by 1.4%.

Target. The ultimate cost of the claims with a loss date inside the row's period, net of excess and recoveries, with repudiated claims left out. Claims are valued at 31 December 2024, so 2024 is not fully reported: only about half of its bodily-injury cost is in. Every model's offset is log(exposure × share of claims reported), with the share taken from a reporting pattern per peril. Open claims are developed with the recovery ratio seen on closed ones, costs are trended to 2025 per peril, and costs above the 99.5% quantile are capped and added back as a load per line (1.02 to 1.06).

Feature selection. A column stays only if it beats 20 random "probe" columns out of time, both on its own and inside a small LightGBM. The pass mark lets at most 1 in 20 noise columns through. Week 1 keeps 53, 44 and 49 features (motor, buildings, contents). Week 2 keeps only 28, 14 and 9, because two years of a small book cannot prove the weaker effects.

2 Week 1: six experts and a geometric mean

For each line I trained six experts, each three times: on 2021-22 to price 2023, on 2021-23 to price 2024, and on all four years to price 2025. All of them are LightGBM with an exact log offset, except the GLM.

Expert	What it models
`total`	Tweedie GBM on capped total cost, times the line's large-loss load
`glm`	Tweedie GLM on banded numeric factors and grouped categories
`peril`	one Tweedie GBM per peril, with its own reporting share and load, summed
`fs_gamma`, `fs_lognorm`	Poisson frequency × claim size (Gamma; or lognormal with a risk-specific variance)
`fs_peril`	frequency × claim size for each peril, summed

The predictions are combined row by row with an equal-weight geometric mean, and each line is then recalibrated with a two-parameter Tweedie GLM, $\log E = \log(\text{exposure} \cdot L) + b_0 + b_1 \log(\text{geo } /L)$, where L is the line's mean rate. The GLM is fitted on the two back-test years; b_1 comes out at 1.04 for motor, 1.10 for buildings and 1.09 for contents.

Back-test D^2, all lines (calibration fitted on the other year)	2023	2024
one LightGBM on total cost	0.1364	0.1338
best single expert (`fs_peril`)	0.1379	0.1365
geometric mean of the six, recalibrated	**0.1386**	**0.1375**

The frequency × claim-size experts beat a single cost model in every line and year, and the six together beat each one of them. A seventh expert, a mixture of experts for claim size, gave the same private score, so I kept the simpler file. The table comes from the rebuilt pipeline. The file I submitted came from the 22 October code, which reported 0.1380 and 0.1367 and was written before the duplicate-claim and years-licensed fixes. Its 2025 predictions have a log correlation of 0.996 to 0.998 with the rebuild.

3 Week 2: adapting to the emerging market

Week 2 differed from Week 1 in most of the ways that matter to a pricing model:

	Week 1	Week 2
History	4 years, 163k claims	2 years, 9k claims (about 1,000 per home line)
New business in the year priced	close to the training mix	77% of 2025, against 12-15% of training rows
Gender	not a rating factor	a permitted rating factor
Weather of the year	inside the training range	2025 far outside it (rainfall anomaly −128 mm against ±3 mm)
Buildings experience	stable	a 2023 fire-cost spike; 2024 came in at 0.59 of 2023's prediction

Transfer from the developed market. I fit a LightGBM on every Week 1 row using only the factors that mean the same thing in both markets (30 for motor, 24 for buildings, 26 for contents), always with renewal count, and put its log rate into the offset of the Week 2 models. The local trees then only have to learn how this market differs, and 20 to 110 trees are enough. Each Week 1 expert is rebuilt the same way, on a prior for its own component (frequency, claim size, each peril). A peril with fewer than 50 claims (300 for claim sizes) uses its prior alone, re-levelled. I dropped the weather of the year.

Choices made by cross-validation. Two years give only one forward test, so the tuning used five folds of customers over 2023-24. It picked one-split trees for motor and contents and a strong GLM ridge; a weak ridge overfits buildings to a D^2 below zero. It also picked a member set per line instead of all six experts (motor `total` + `fs_peril` + `glm`, buildings `fs_lognorm` + `peril`, contents `total` + `glm`), because two years hold too few claims to learn claim sizes. The recalibration slope stays at 1, since a slope fitted on one year did not carry to the other. New-business and gender factors are measured on out-of-fold predictions and shrunk towards no effect.

Forward test: fit on 2023, score 2024, all lines	D^2
LightGBM on Week 2 data only	0.1026
developed-market prior alone	0.1259
LightGBM on top of the prior	0.1277
ensemble on priors	**0.1300**

4 What the leaderboard taught me

The ensemble (v4) scored 0.16198 in public. My earlier single transfer LightGBM had scored 0.16529, even though the ensemble ranked the 2024 risks better. The gap came from the levels. The 2024 back-test is 13% new business and 2025 is 77%, so the back-test can judge the ranking inside a segment but not the level of new business, buildings or gender.

So I treated ranking and level as two separate problems. For the ranking, v5 keeps the earlier file's total in each of ten segments (line × new business × gender) and replaces the ranking inside them with a 50/50 geometric blend of that file and my ensemble. On 2024

rows weighted to the 2025 mix, the blend ranks better (within-segment D² 0.1067 against 0.1016). v5 changed a quarter of the rows by more than 10% and still scored exactly 0.16529. That surprised me, and it settled the question: the public score hardly reacts to the ranking inside segments.

Levels. For Tweedie p = 1.5, multiplying a segment's predictions by c changes the deviance by exactly

$$\Delta D = 4\left[(c^{-1/2}-1)A + (c^{1/2}-1)B\right], \qquad A = \sum y\,\mu^{-1/2}, \quad B = \sum \mu^{1/2},$$

so the best level is $c^* = A/B$, and any scored file that differs from another only by segment factors is a measurement of it. I fit one model, linear in the ten best levels, to 13 public scores. It matches all of them within 0.0001, and when five were left out of the fit it predicted them up to one constant offset. It says motor needs a much wider gender split than any model of 2023-24 gave: new-business men about ×1.37 on v5, women about ×0.97 (Figure 2).

Luck. The public score covers only a sample of the 2025 rows, so a segment's best level on that sample is off by $\mathrm{sd} = \sqrt{\phi/(\mathrm{share}\cdot\sum \mu^{1/2})}$. At a 30% public share that is 8% for motor new-business men, 13% for buildings new business and 24% for buildings renewals. The model fits this luck as a separate term and gives two answers: the best levels for the public rows (v13), and the best levels for the rows not yet scored (v12), which pulls the noisy segments back towards v5.

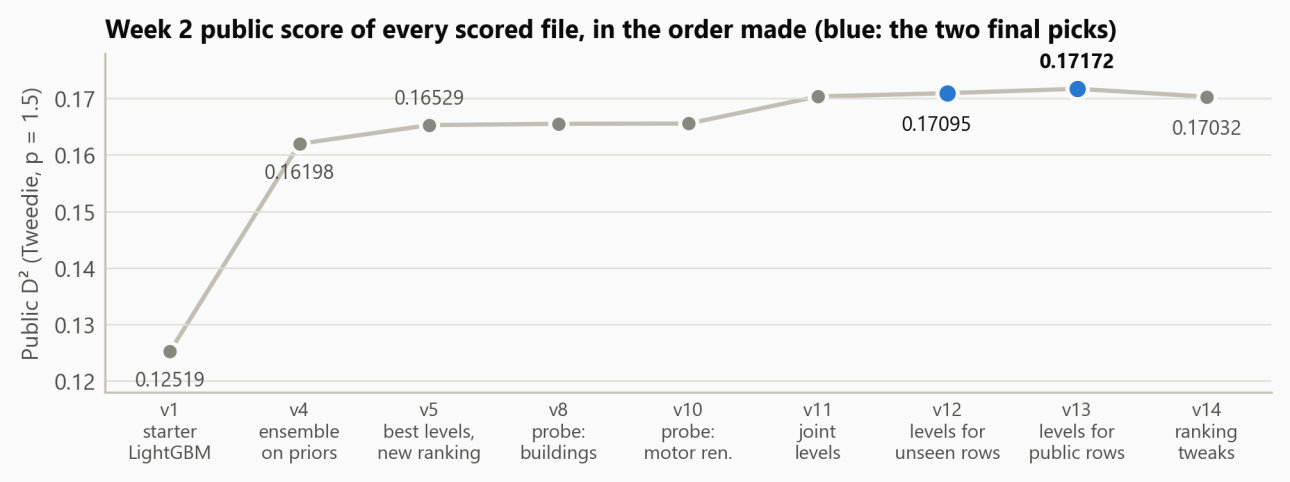


Figure 1: Every scored Week 2 file, in the order made. Changing levels moved the public score; changing the ranking (v5, v14) hardly did.

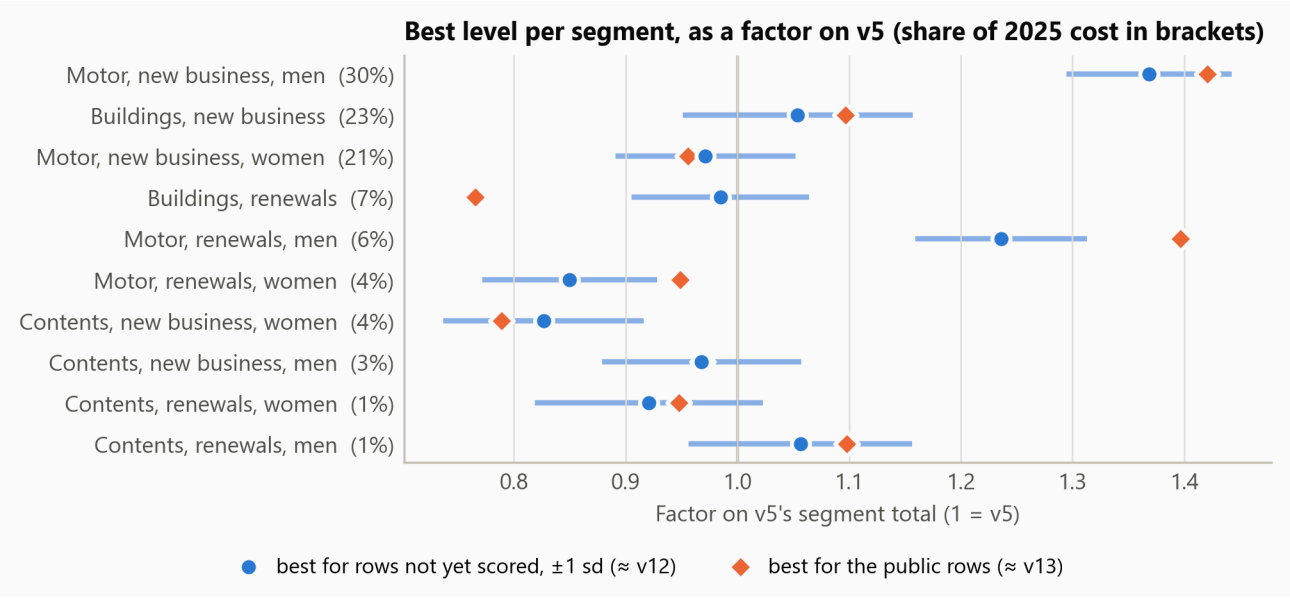


Figure 2: Best level per segment. v12 targets the blue points, v13 the orange ones. They differ most where the public rows say least (buildings renewals, motor renewal men).

v12 scored 0.17095 (expected 0.1708) and v13 0.17172 (expected 0.1714). Against v5, the model expects v12 to gain 0.0042 on unseen rows and v13 0.0037, with an sd of about 0.0015. I selected both on Kaggle. v12 is my best estimate for the final ranking;

v13 only wins if the true levels are as extreme as the public rows suggest. The Week 2 file in this submission is v13, my best score. I also tried two last ranking changes: no this-year weather in the buildings and contents model, and a motor spread of ×1.05. That file, v14, scored 0.17032, which is 0.0006 below v12 against a public-luck sd of 0.0008. The leaderboard cannot tell whether that is a real loss, so I did not select it.

What the new market changed. I stopped learning everything from local data: the developed market supplies the relativities, and the emerging data corrects them. Simpler local models did better (one-split trees, a strong ridge, fewer experts, no fitted slope). The back-test was no longer enough, because 2025 is mostly new business, so I judged ranking and level on different evidence. And I treated the public leaderboard as a noisy measurement to model rather than a score to push up.

5 Major decisions

Decision	Rejected	Evidence
Fix data errors with evidence; drop and impute nothing	dropping or imputing rows	cleaning log per round
Geometric mean of experts, recalibrated per line (Week 1)	one LightGBM	+0.0022 (2023) and +0.0037 (2024) back-test D^2
Developed-market prior in the offset (Week 2)	Week 2-only models	+0.025 forward D^2
Members per line by cross-validation; slope fixed at 1	all six experts; fitted slope	contents: all six 0.1071 against `total` alone 0.1087
Segment levels kept apart from ranking	trusting the back-test's levels	v4 0.16198 against v5 0.16529
Aim at unseen rows rather than the public sample	chasing the public score	public luck sd 8% to 24% per segment

6 Use of AI

I worked alone, with an AI coding agent (Claude Code in VS Code) at every step. The agent wrote and ran most of the code and drafted this write-up. I set the questions and decided what got built.

Most of the work was a long run of prompts that proposed ideas and tested them: the developed-market prior, the expert ensembles, the segment-level probes, the public-luck model. Each idea became a script and had to hold up out of time before it counted.

The prompts carried context that pointed the agent somewhere specific: the case study, the question to answer, and what had already failed and why, which I kept in `KNOWLEDGE.md`. That kept it from starting over or retrying dead ends.

I used different skills for different jobs. Two project skills in the repository (`.claude/skills/data-scientist` and `ml-engineer`) set how the agent writes code and runs an analysis: frame the problem first, test out of time, distrust its own gains, and keep the code reproducible and under version control.

I did not accept a result from the same run that produced it. Each idea was questioned separately, so the agent had to check its own work, and this caught real problems. Factor searches looked strong when the same two years both chose and scored the factors; a permutation test then showed that real factors pass no more often than shuffled ones, and only gender survived. A later review found a sign error in the Week 1 contents frequency trend. It is documented, left in the code that reproduces the submitted file, and fixed in the rebuild.

Nothing went into the pipeline or a submission until I had reviewed the plan and its evidence. I chose the target definition, what counted as evidence, which files to submit and select, and when to stop probing the leaderboard.

7 Assumptions

- 2025 claims follow 2023-24 after trend (motor severity 0 to 7.9% a year depending on peril, buildings severity 6%, contents frequency 1.4% and severity 3.4%). Reporting patterns are stable, and open claims develop the way closed ones did.
- For factors defined the same way in both markets, the developed market's relativities are a reasonable starting point. The emerging market differs in level and in a few effects that the local models can learn.
- The public leaderboard rows are a random sample of 2025, about 30%. The order of v12 and v13 holds for any share from 20% to 50%.
- Gender is a permitted rating factor in the emerging market, as the case study states.

8 Limitations

- Buildings is the part I trust least. There are only two years of emerging-market data, a few large claims decide the buildings public score, and its levels carry 8% to 10% uncertainty.
- The Week 2 levels rest partly on the public leaderboard. The competition allows that, but in real pricing it would mean calibrating on a sample of the book's own outcomes, and the bands in Figure 2 are wide.
- Back-tests cannot judge new-business levels, and ranking changes such as v14 are smaller than the noise in the public score.
- The submitted Week 1 file still has the contents trend sign error. The per-line recalibration absorbs most of it (the intercept is about +10%), but the contents relativities may be slightly off.

9 Guardrails

- Columns driven by the outcome are excluded: `renewal_decision`, and `customer_service_contacts`, which is zero on 0.2% of rows with a claim against 67% of rows without one. Claims history is built as it was known on 1 January of each year, so no feature looks ahead.
- The written premium is excluded, because a model using it would copy the old tariff. So are marketing and service fields and personal data such as surname. Gender is used only where it is permitted, and its factor moves price between men and women without changing a line's total.
- Every file is tested out of time before it is submitted, then registered as a new numbered version with its evidence, description and checksum. Versions are never overwritten.
- A file is saved only if it has one row per sample row id, in the sample's order, with every prediction finite and positive. The Week 1 code adds block and escalate rules: a back-test D^2 at or below zero blocks the file; an A/E outside 1 ± 5%, feature drift with a PSI of 0.25 or more, or a pure-premium shift of more than 15% against history goes to actuarial sign-off.
- Probe columns in feature selection, permutation tests on factor searches and the public-luck model are all there to stop me fitting noise, including apparent gains that an AI agent reports with confidence.
- Every step is a script under git, and `final/pipeline.py` runs them in order. From the intermediate results in the repository it rebuilds the Week 2 file byte for byte and the Week 1 file to a relative difference of 3e-15, in about a minute.

Appendix (outside the five graded pages)

A. Week 2 submission register (from `2nd round/submissions/versions.csv`).

Version	Public D^2	What it is
v1	0.12519	starter: one LightGBM per line, no prior, no trend or large-loss treatment
v2	not scored	LightGBM on the developed prior, with new-business and gender factors
v3	not scored	ensemble on developed priors, recalibration slope fitted (kept for comparison)
v4	0.16198	ensemble on developed priors, level only (slope 1)
v5	0.16529	earlier best file's segment totals; ranking 50% that file, 50% ensemble
v6	not scored	as v5, ranking 25% / 75%
v7	not scored	probe: v5 with motor new business x1.15
v8	0.16552	probe: v5 with buildings new business x1.08
v9	not scored	probe: v5 with contents new business x1.10
v10	0.16556	probe: v5 with motor renewals x1.07
v11	0.17036	v5 with every segment at its best level from all scores
v12	0.17095	v5 with every segment at its best level for rows not yet scored
v13	0.17172	v5 with every segment at its best level for the public rows (Week 2 file)
v14	0.17032	v12's levels; buildings and contents ranked without this-year weather; motor spread x1.05
v15	not scored	v12 with the motor spread x1.05 only

B. Segment levels (from `2nd round/reports/11_joint/levels.csv`; factors on v5).

Segment	Share of 2025 cost	Public-row luck sd	Best for public rows	Best for unseen rows	sd
Motor, new business, men	30.2%	8.3%	1.421	1.369	0.074
Buildings, new business	23.3%	12.8%	1.097	1.054	0.103
Motor, new business, women	21.5%	9.0%	0.956	0.971	0.081
Buildings, renewals	6.7%	23.5%	0.765	0.985	0.079
Motor, renewals, men	5.8%	16.8%	1.397	1.236	0.077
Motor, renewals, women	4.1%	18.2%	0.949	0.850	0.078
Contents, new business, women	3.8%	9.8%	0.789	0.826	0.090
Contents, new business, men	3.1%	10.2%	0.969	0.968	0.089
Contents, renewals, women	0.9%	18.6%	0.948	0.921	0.102
Contents, renewals, men	0.8%	19.5%	1.098	1.057	0.100

C. Files and how to reproduce. The two prediction files, their checksums and sources are in `final/1_predictions/manifest.csv`. The code and data are at https://github.com/IKSHRESTHA/IAA-Hackathon. `python final/pipeline.py` reruns the last stages of both pipelines from the intermediate results in the repository (about a minute) and checks the rebuilt files against the submitted ones; `--full` reruns every stage from the raw competition files. `final/README.md` lists the stages.